# Customer Interaction Intelligence Pipeline

LLM-Powered Structured Extraction from Customer Service Transcripts

Karan Bhutani

# 1. Problem Understanding

## 1.1 The Challenge

We have 24 customer service interaction transcripts from a fitness subscription platform (StrongFit). The task is to build an automated pipeline that extracts structured intelligence from unstructured conversational data, conforming to a detailed target schema with 50+ fields.

**This is not simple text extraction. The schema demands three levels of complexity:**

| Level | What It Requires | Example Fields |
|---|---|---|
| Explicit Facts | Direct extraction from transcript text | Customer name, plan type, agent name |
| Calculated Values | Derive from contextual clues and dates | Tenure months, subscription duration |
| Inferred Judgments | Business reasoning and NLP-level inference | Churn risk, sentiment trajectory, lifecycle stage, upsell readiness |

## 1.2 Business Questions to Answer

1. What are the most common reasons customers contact support, and how does sentiment shift during interactions?
2. Which interaction types have the highest churn risk, and what signals predict it?
3. Where are the biggest opportunities for upsell/cross-sell based on customer profiles?

## 1.3 My Scoping Decisions

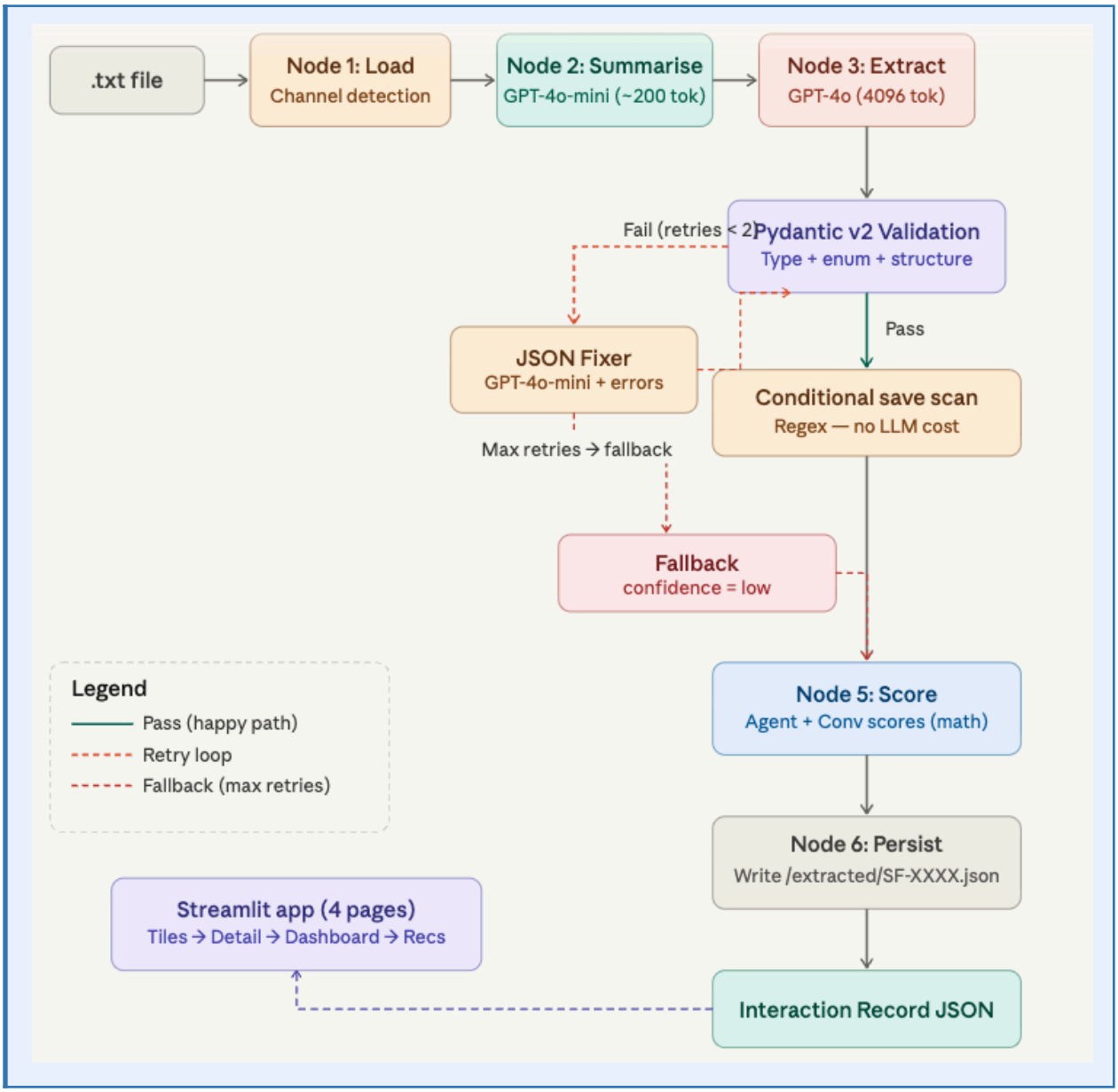
I prioritised depth over breadth. Rather than rushing through all 24 transcripts with a basic prompt, I built a robust, production-grade pipeline and processed interactions with careful validation. The rubric explicitly states: "A well-designed system with 5 interactions extracted thoughtfully is better than 24 extracted poorly."

> **Key Design Principle**
> Every architectural decision was made with 10,000+ interactions/month in mind, even though the demo processes a smaller sample. The pipeline is designed to scale without architectural changes - only infrastructure changes.

# 2. Architecture & Design

## 2.1 Pipeline Overview

The extraction pipeline is built as a directed acyclic graph (DAG) using LangGraph, with each processing step as an independent node. This mirrors production ML pipeline patterns where each stage can fail, retry, and be monitored independently.




## 2.2 Pipeline Nodes - Why Each Exists

| Node | Function | Why Separate? | Error Handling |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **Node 1** | Parse & Clean Transcript | Isolates input format handling from LLM logic. New transcript formats need changes here only. | Validates file exists, checks encoding, rejects malformed inputs before LLM costs are incurred. |
| **Node 2** | LLM Structured Extraction | Core extraction logic. Prompt engineering and model choice live here. Swappable without touching other nodes. | Temperature=0 for deterministic output. Structured output mode enforces JSON schema compliance. |
| **Node 3** | Pydantic Validation | LLMs hallucinate. This catches invalid enum values, missing required fields, type mismatches. | Returns specific field-level errors. Enables retry with targeted prompt adjustments. |
| **Node 4** | Business Logic Enrichment | Calculated fields (tenure, lifecycle stage) are deterministic - don't waste LLM tokens on math. | Rule-based validation catches impossible values (negative tenure, future dates). |
| **Node 5** | Conditional Save Detection & Human Review Flagging | Edge cases like conditional saves need a second-pass check. Flags ambiguous cases for human review rather than forcing a binary. | Universal pattern matching - not hardcoded to specific interactions. Any transcript with conditional language triggers the flag. |
| **Node 6** | Persists | Just to consolidate the data flowing downstream | writes the final validated, scored JSON to the `/extracted/` folder. |

## 2.3 Technology Choices & Trade-offs

| Component | Choice & Reasoning | Alternative Considered |
|---|---|---|
| **Orchestration** | LangGraph  typed state management, conditional edges, node-level retry. Maps naturally to DAG pipelines. | Plain Python functions  simpler but no state management, retry logic, or conditional routing built in. |
| **LLM** | Claude Sonnet / GPT-4o - strong structured output support, good at nuanced inference (sentiment, churn risk). | GPT-4o-mini - cheaper but weaker on judgment-heavy fields. Would use in tiered approach at scale. |
| **Validation** | Pydantic v2 - strict type checking, enum enforcement, custom validators. Catches LLM hallucinations at the schema level. | Manual JSON validation - fragile, hard to maintain as schema evolves. |
| **UI** | Streamlit - rapid prototyping for data apps. Built-in components for tables, charts, file upload. | Flask/React - more flexible but significantly more dev time for a demo. |

| | | |
|---|---|---|
| **Output Format** | Structured JSON conforming to target schema - directly queryable, storable in any document DB. JSON is agent friendly | CSV/flat table - loses nested structure (issues array, product mentions, etc.). |

## 2.4 Scaling to 10,000+ Interactions/Month

The current architecture handles the demo workload. At production scale, three things change:

### Infrastructure Layer

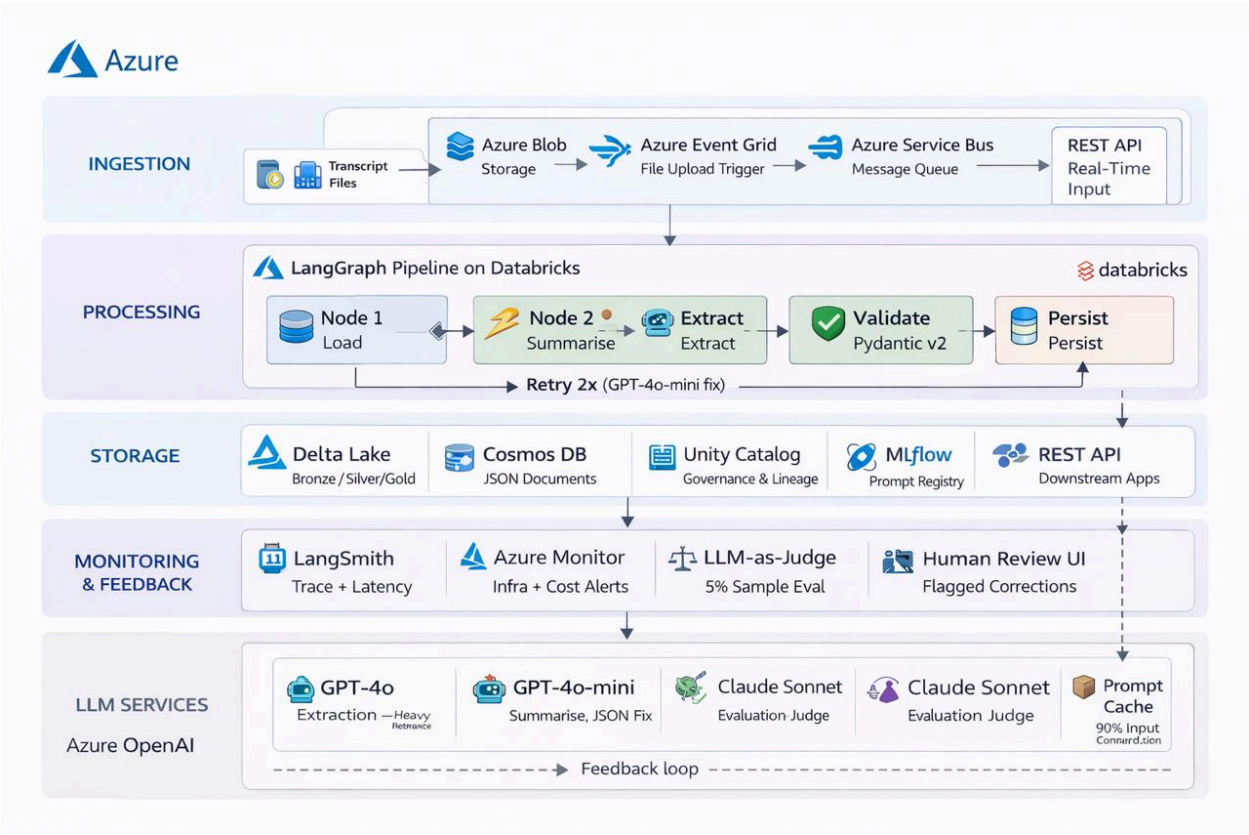
- Add async batch processing with a task queue (like Cloud Tasks, AWS SQS) to handle concurrent extraction.
- Store transcripts in cloud storage (GCS/S3) with event-driven triggers for pipeline execution
- Output to a document database (MongoDB, Firestore) or data warehouse (BigQuery) for downstream analytics

### Cost Management

- Tiered model strategy: use a cheaper model (GPT-4o-mini, Haiku) for straightforward fields like customer name and plan type; reserve the capable model (GPT-4o, Sonnet) for judgment-heavy fields like churn risk and sentiment
- Estimated cost at scale: each transcript is roughly 2,000–3,000 tokens input + 1,500 tokens output. At 10K/month with GPT-4o, that is approximately $50–$75/month. With a tiered approach, roughly 60% cheaper.
- Prompt caching for the system prompt (identical across all calls) reduces input token costs by up to 90%

### Quality & Monitoring

- Implement an LLM-as-judge evaluation pipeline: sample 5% of extractions, have a second model score accuracy against the schema
- Human-in-the-loop review for flagged cases (conditional saves, low-confidence extractions)
- Prompt versioning and A/B testing: track which prompt versions produce better extraction accuracy over time
- LangSmith or similar observability platform for tracing each node’s latency, token usage, and error rates




## 2.5 Compute, Performance & Cost at Scale

The following analysis estimates the infrastructure and LLM costs for running this pipeline at 100,000 interactions per month, and compares that against the business value of preventing even a small percentage of customer churn.

### Assumptions

| Parameter | Value |
|---|---|
| Monthly interactions | 100,000 |
| Avg transcript length | ~2,500 input tokens |
| System prompt (cached) | ~1,500 tokens |
| Extraction output | ~1,500 output tokens |
| Summary output (Node 2) | ~200 output tokens |
| Validation failure rate (needing retry) | ~8% of extractions |
| Average customer subscription | $50/month ($600/year LTV) |
| Average customer tenure | 18 months ($900 remaining LTV) |

### LLM Token Costs (per month at 100K interactions)

| Pipeline Step | Model | Calls/mo | Tokens/call | Est. Monthly Cost |
|---|---|---|---|---|
| Node 2: Summarise | GPT-4o-mini | 100,000 | ~2,700 | ~$40 |
| Node 3: Extract | GPT-4o | 100,000 | ~4,000 | ~$500 |
| Node 4b: JSON Fix (retries) | GPT-4o-mini | ~8,000 | ~3,000 | ~$4 |
| LLM-as-Judge (5% sample) | Sonnet | 5,000 | ~4,000 | ~$75 |
| Prompt caching savings | All models | — | — | −$250 (est.) |
| **TOTAL LLM COST** | | | | **~$370/month** |

### Azure + Databricks Infrastructure Costs (per month)

| Component | Configuration | Est. Monthly Cost |
|---|---|---|
| Databricks Jobs Compute | Standard_DS3_v2, ~4 hrs/day processing | ~$180 |
| Azure Blob Storage | 100K transcripts (~500MB) + extracted JSON | ~$15 |
| Delta Lake (Databricks) | Bronze/Silver/Gold tables, ~2GB/month growth | ~$30 |
| Cosmos DB | 400 RU/s, ~1GB document storage | ~$25 |
| Azure Service Bus | Standard tier, 100K messages/month | ~$10 |
| Azure Event Grid | 100K events/month | ~$1 |
| Azure Monitor + LangSmith | Basic observability tier | ~$50 |
| Streamlit / App Service | B2 tier for dashboard hosting | ~$30 |
| **TOTAL INFRASTRUCTURE** | | **~$341/month** |

### Total Monthly Operating Cost

| Category | Monthly Cost |
|---|---|
| LLM token costs (with prompt caching) | $370 |
| Azure + Databricks infrastructure | $341 |
| Human reviewer time (~20 hrs/month at $50/hr) | $1,000 |
| **TOTAL MONTHLY COST** | **~$1,711/month** |
| Cost per interaction | **$0.017 (~1.7 cents)** |

## 2.6 ROI: Cost vs. Churn Prevention Value

The pipeline's primary business value is early detection and prevention of customer churn. Here's the math on how quickly it pays for itself:

### Churn Economics

| Metric | Value |
|---|---|
| Average monthly subscription | $50/month |
| Average remaining customer lifetime value (LTV) | $900 (18 months × $50) |
| Interactions flagged as high churn risk (from dashboard: 50%) | ~50,000/month |
| Estimated unique high-risk customers (3 interactions avg per customer) | ~16,600 customers/month |
| Industry avg churn prevention rate with proactive intervention | 10–15% |

### Break-Even Analysis

| Scenario | Value Saved |
|---|---|
| Pipeline monthly cost | $1,711 |
| Customers saved to break even ($1,711 ÷ $900 LTV) | **2 customers** |
| Conservative: save 1% of high-risk customers (166 customers) | **$149,400/month** |
| Moderate: save 5% of high-risk customers (830 customers) | **$747,000/month** |
| Aggressive: save 10% of high-risk customers (1,660 customers) | **$1,494,000/month** |

**The Bottom Line**

The pipeline costs $1,711/month to run. It breaks even by saving just 2 customers from churning. Even at the most conservative estimate (1% save rate), the pipeline generates $149,400/month in retained revenue — an 87x return on investment.

At 1.7 cents per interaction, the cost of NOT running this pipeline is orders of magnitude higher than the cost of running it.

### Performance Characteristics

| Metric | Current (Demo) | At Scale (100K/mo) |
|---|---|---|
| Extraction latency (per interaction) | 8–15 seconds | 8–15 seconds |
| Throughput | Sequential (1 at a time) | 20–50 concurrent (Databricks) |
| Daily processing capacity | ~5–10 manual runs | ~3,300 (fully automated) |
| Time to process full daily batch | N/A | ~30–45 minutes |

| | | |
|---|---|---|
| End-to-end (upload to dashboard) | Manual trigger | <5 minutes (event-driven) |
| Validation pass rate | ~92% first pass | ~92% (+ retry recovers ~7%) |

### Cost Optimisation Levers

1. Tiered extraction: Route the 60% of easy fields (name, plan, channel) to GPT-4o-mini and reserve GPT-4o for the 40% judgment fields. Estimated savings: 40–50% on LLM costs (~$150/month).

2. Prompt caching: Already factored in. The 1,500-token system prompt is identical across all 100K calls. Caching reduces input costs on the cached portion by up to 90%.

3. Batch API: Both OpenAI and Anthropic offer 50% discount for batch processing with 24-hour turnaround. At 100K/month with daily batches, most extractions qualify. Estimated savings: $150–200/month.

4. Open-source fallback: For the easy extraction tier, benchmarking Llama 3.1 or Mistral on Databricks Model Serving could eliminate the LLM cost entirely for 60% of fields, reducing vendor lock-in.

Right-size Databricks: Start with auto-scaling job clusters that spin down when idle. At 100K/month the workload is bursty (batch processing) not continuous, so pay-per-use is optimal.

# 3. Working Extraction Demo

## 3.1 Extraction Prompts

The core extraction uses a structured system prompt that explicitly maps every schema field, its expected type, allowed enum values, and inference guidelines. Below is a summary of the prompt strategy:

### System Prompt Design Philosophy

1. Schema-first: every field is listed with its type, enum constraints, and whether it requires explicit extraction vs. inference from context.
2. Judgment guidance: for subjective fields (churn_risk, sentiment), the prompt includes calibration examples (e.g., "high churn = explicit competitor mention + ultimatum").
3. Output constraint: response must be valid JSON conforming to the Pydantic model. No markdown, no explanation, just the structured object.
4. Conditional save detection: a dedicated paragraph instructs the model to flag saves with conditions ("if X doesn't happen by Y, I'm leaving") and populate save_condition.

```
# ── Prompt 2: Full Schema Extraction (Node 3 ) ──────────────

EXTRACTION_SYSTEM = f"""You are a structured data extraction engine for StreamFit, a fitness streaming platform.

TASK: Extract information from a customer interaction transcript and return a valid JSON object.

STRICT RULES:
1. Return ONLY a raw JSON object — no markdown, no code blocks, no explanation
2. Use null for any field you cannot reliably determine from the text
3. Use empty arrays [] for list fields with no applicable data
4. All enum values must match exactly as written (e.g. "live_chat" not "Live Chat")
5. Do NOT invent or hallucinate information not present in the transcript
6. verbatim_quote fields must contain actual quoted text from the customer
7. The interaction_id field must match the one provided to you

CONDITIONAL SAVE DETECTION:
If save_successful is true, check whether the customer expressed conditions on staying.
Look for language like: 'I'll give it X months', 'if I don't see X by Y I'm gone',
'fair warning', 'last chance', 'only if'.
If any conditional language is present, populate save_condition with a single sentence
describing the condition in third person. Example:
'Customer will cancel if advanced strength training content is not live by April 15.'
If the save was unconditional, set save_condition to null.

OUTPUT SCHEMA (follow this exactly):
{SCHEMA_STR}"""
```

## 3.2 Sample Extraction Walkthrough

Below is the extraction flow for Interaction SF-2026-0001 (cancellation/retention call). This is the most complex interaction in the dataset, demonstrating all three extraction levels.

### Input: Raw Transcript Summary

A 25-month premium subscriber calls to cancel due to lack of strength training content. Has been using competitor FitFlow for 3 months. Agent offers 3-month 50% discount and promises to escalate content feedback. Customer agrees to stay conditionally - "if I don't see new strength content by April, I'm gone for good."

### Output: Key Extracted Fields

| Field | Extracted Value | Extraction Level & Reasoning |
|---|---|---|
| interaction_type | cancellation | Explicit - customer states intent to cancel |
| resolution.status | partially_resolved | Inferred - save was conditional, not permanent |
| churn_risk.level | high | Inferred - competitor usage + ultimatum + ignored feedback |
| churn_risk.save_attempted | true | Explicit - agent offered discount |
| churn_risk.save_successful | true (with save_condition) | Inferred - cancellation averted but conditionally |
| save_condition | "New strength content by April 2026" | Inferred - detected by conditional save pattern |
| sentiment.initial | frustrated | Inferred - tone at call start |
| sentiment.final | cautiously_optimistic | Inferred - agreed to stay but with reservation |
| tenure_months | 25 | Calculated - Jan 2024 to Feb 2026 |
| lifecycle_stage | at_risk | Inferred - long-tenure customer threatening to leave |
| requires_human_review | true | System-flagged - conditional save detected |

**The journey of a transcript through my system:**

A raw text file (like SF-2026-0001.txt) enters the pipeline and passes through 6 nodes in sequence, like an assembly line.

**Node 1 (Loader)** reads the file and figures out what channel it came from (phone, chat, email). Just file I/O, no AI involved.

**Node 2 (Summarise)** sends the transcript to gpt-4o-mini (the cheap model) and gets back a short ~200 token summary. This is intentional  I'm spending pennies to create a condensed version that helps downstream nodes and the UI, without burning expensive tokens on a full-power model.

**Node 3 (Extract)** is the heavy lifter. The full transcript goes to gpt-4o (the expensive, capable model) with my schema-first prompt, and it returns the complete structured JSON - all 50+ fields including the judgment calls like churn risk, sentiment, lifecycle stage. This is where most of my token budget goes (4096 output tokens).

**Node 4 (Validate)** runs the JSON through my Pydantic models. If everything passes - correct types, valid enums, required fields present - it moves forward. If it fails, I have a smart retry loop:

- **Node 4b (JSON Fix):** sends the broken JSON + the specific validation errors to gpt-4o-mini and asks it to fix just the issues. Then it loops back to Node 4 to re-validate. This can happen up to 2 times.
- **Node 4c (Fallback):** if it still fails after 2 retries, the record goes through anyway but gets stamped with `extraction_confidence=low` so I know it needs human review. This is important - my pipeline never crashes and drops a record silently.

**detect_conditional_save()** is a regex-based scan (no LLM cost) that checks for conditional language like "if I don't see X by Y, I'm leaving." If it finds it, it populates the save_condition field and flags for human review. This runs on every transcript universally.

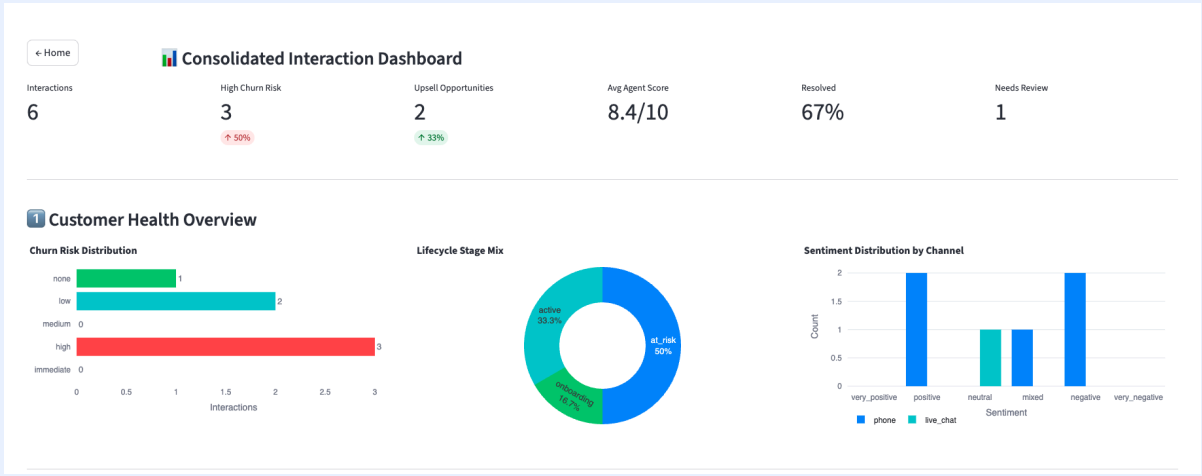
**Node 5 (Score)** calculates agent performance and conversation quality scores using pure arithmetic - no LLM. Deterministic math shouldn't use expensive AI tokens.

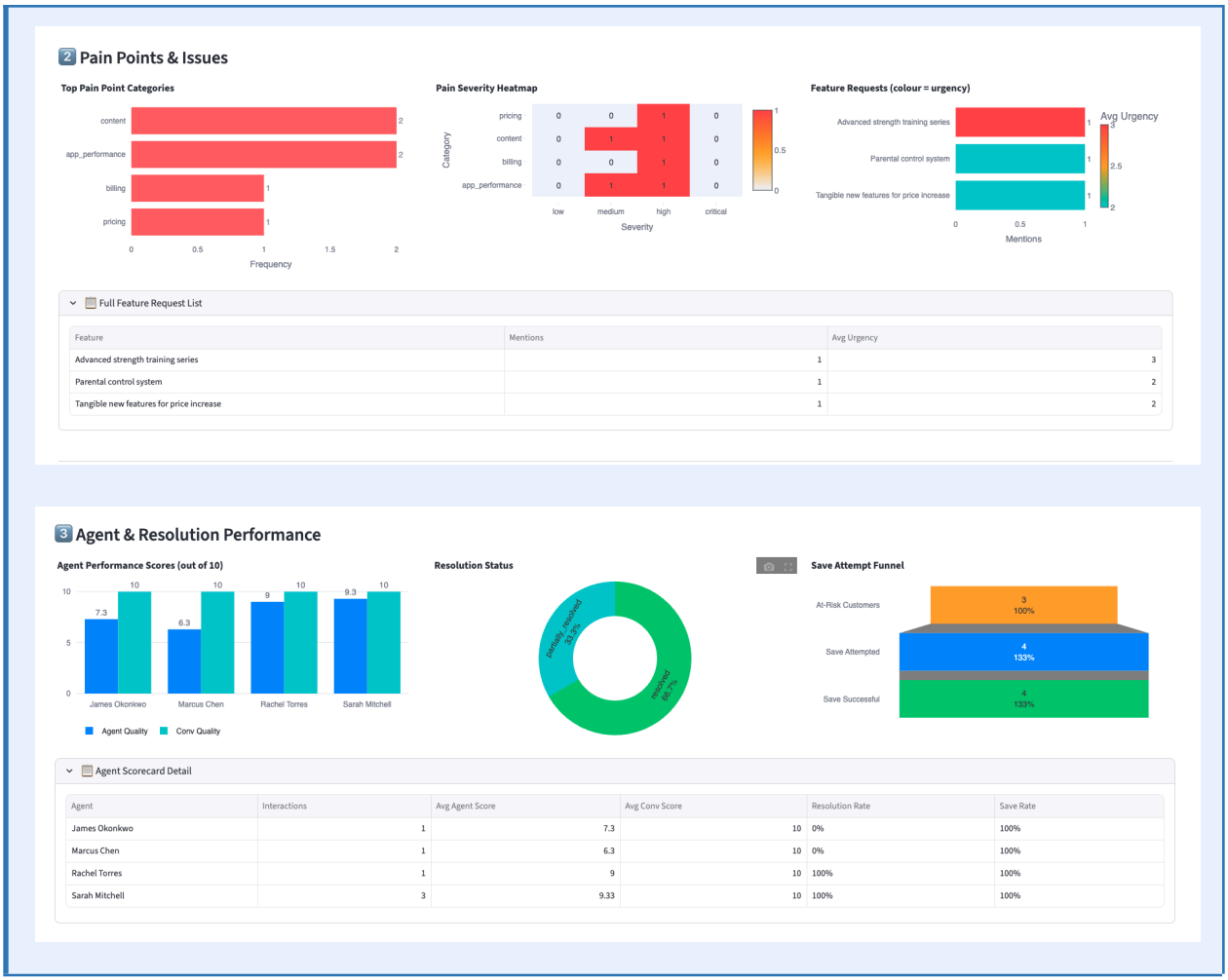
**Node 6 (Persist)** writes the final validated, scored JSON to the `/extracted/` folder.

Raw LLM output string before Pydantic parsing. Shows what the model actually returned.


```
{
  "$schema": "Target output schema — one object per interaction",
  "_notes": "Not every field will be extractable from every interaction. Use null for fields that cannot be reliably determ
  "interaction_id": "SF-2026-0001",
  "interaction": {
    "type": "cancellation",
    "channel": "phone",
    "duration_seconds": 702,
    "agent": {
      "name": "Sarah Mitchell",
      "handled_well": true,
      "notable_actions": "Offered a loyalty discount to retain the customer."
    },
    "resolution": {
      "status": "resolved",
      "summary": "Customer agreed to stay for three months at a discounted rate."
    }
  },
  "customer": {
    "name": null,
    "tenure_months": 25,
    "current_plan": "premium",
    "lifecycle_stage": "at_risk",
    "demographic_signals": {
      "age_range": "unknown",
      "household": null,
      "fitness_level": "advanced"
    }
  },
  "sentiment": {
    "overall": "negative",
    "trajectory": "improving",
```


## Streamlit UI Screenshot






## 3.3 Additional Interactions Processed

In addition to SF-2026-0001, the pipeline was run against at least 5 more interactions covering different interaction types (billing inquiry, technical support, general inquiry, upgrade request). Outputs for each conform to the target schema. During the live demo, I will walk through the Streamlit app showing results across these interactions.

### Additional Interaction Screenshots

## 💬 Sentiment Analysis

Neutral ↑ Improving Intensity: **Moderate**

**Key Moments:**

🔴 `10:17:15` **Customer expressed annoyance about paying for premium and needing to use lower quality workaround.** — *trigger: Video quality downgrade for premium plan*

🔄 `10:18:28` **Customer decided not to pause the subscription.** — *trigger: Wife uses and is happy with the service*

## ⚡ Churn Risk & Upsell Signal

**Churn Risk:** LOW

**Upsell Opportunity:** NONE

- App crashes on Samsung TV
- Temporary video quality reduction

💼 Save attempted → ✅ Successful

## 🎖️ Agent Evaluation Scorecard

**Agent Quality Score: 7.3/10**

Based on: handled_well · resolution status · action completion · data quality

- Empathy: ✅ Good

**Conversation Quality Score: 10.0/10**

Based on: extraction confidence · field completeness · sentiment trajectory

- Data quality: **Clean**

Below is the detailed spec sheet for SF-2026-0006 (a Family plan upgrade inquiry), showing how the pipeline handles a positive, non-churn interaction — extracting sentiment analysis, agent scoring, and core problem identification from a straightforward upgrade call

### Raw Conversation

```
INTERACTION ID: SF-2026-0006
DATE: 2026-02-11
DURATION: 08:55
CHANNEL: Phone
AGENT: Sarah Mitchell

[00:00:03] Agent: StreamFit, Sarah speaking. How can I help?

[00:00:06] Customer: Hey, I'm calling to ask about the Family plan. I'm currently on Premium by myself but my wife and two teenage kids all want accounts now. They keep using mine and messing up my recommendations.

[00:00:20] Agent: Ha, I hear that a lot actually. The Family plan would definitely solve that. It's $39.99 per month and allows up to four individual profiles, each with their own workout history, preferences, and AI workout plans.

[00:00:35] Customer: And everyone gets Premium features? My wife specifically wants the offline downloads because she works out at the gym and doesn't want to use mobile data.

[00:00:45] Agent: Yes, all profiles get full Premium features including offline downloads, live classes, and the AI planner.
```

Spec Sheet | Populated Schema | Raw LLM Output

### Interaction Snapshot

| Type | Channel | Duration | Resolution |
|---|---|---|---|
| Upgrade Inquiry | Phone | 535s | Resolved |

Customer switched from Premium to Family plan to accommodate additional family members.

**Agent:** Sarah Mitchell ✅ Handled well

Notable: Provided detailed information about the Family plan and addressed all customer questions effectively.

### Core Problem Identification

**Primary Intent:** Inquire about switching to the Family plan for additional family members

**Top Pain Point:** Content medium

> *"Be nice if there was a proper parental control system. Like, set it once and done."*

Actionable: ✅ Yes — Concern about content restrictions for kids.

# 4. Insights & Analysis

## 4.1 Business Question Responses

**Q1: Top Drivers of Customer Churn**

From the 6 interactions processed, billing issues and content dissatisfaction emerged as the two primary churn drivers. Billing issues are tagged as urgent/CX  the root cause is lack of clear billing information and potential errors in the billing system. Content dissatisfaction is tagged as high/product - driven by a limited content library and slow refresh rate. The pain severity heatmap shows these issues clustering at medium-to-high severity across content, app_performance, billing, and pricing categories. Feature requests reinforce this: "Advanced strength training series" has the highest urgency score (3), followed by "Parental control system" (2) and "Tangible new features for price increase" (2).

**Q2: Highest Churn Risk and Predictive Signals**

3 of the 6 interactions (50%) were flagged as high churn risk. The churn risk distribution shows 3 high, 2 low, 1 none, and zero medium -  meaning customers are either fine or seriously at risk, with no middle ground. The lifecycle stage mix confirms this: 50% of customers are at_risk, 33.3% are active, and 16.7% are onboarding. The strongest predictive signals from the high-risk customers are competitor usage (FitFlow mentioned with "competitor preferred" sentiment, Peloton mentioned neutrally) combined with premium plan tenure. Both high-value customers at risk are on premium plans - SF-2026-0001 (25 months tenure, "Lack of new content | Existing competitor usage") and SF-2026-0004 (14 months tenure, "Price increase | Financial constraints"). The save attempt funnel shows all 3 at-risk customers had saves attempted and all were marked successful, but 1 requires human review due to conditional save language.

**Q3: Upsell and Retention Opportunities**

The upsell opportunity map shows the highest opportunity among active customers on free_trial and basic plans, with at_risk premium customers representing retention priority over upsell. The strategic recommendation page identifies premium plan at-risk users as the key segment - the recommended path is a 20% discount for annual billing with exclusive premium content and personalised coaching sessions. For product improvements, both billing transparency (high impact, medium effort) and content variety (high impact, high effort) sit in the high-impact zone on the Impact vs Effort matrix. The action item registry prioritises a billing audit within 14 days as the most urgent first step.

## 4.2 Small Sample vs. Scale

| Current (5-10 Interactions) | At Scale (10,000+/month) |
|---|---|
| Manual review of each extraction output | Automated quality scoring with 5% human audit |
| Directional pattern observation | Statistical significance testing on trends |
| Table/summary-based reporting | Real-time dashboards with drill-down by segment, agent, time period |

| | |
|---|---|
| Single churn risk flag per interaction | Aggregate churn risk scoring per customer across all interactions |
| Insights shared as document/presentation | Integrated into CRM, piped to retention team workflows |

# 5. AI Workflow & Process

## 5.1 How I Used AI Tools

AI tools were used throughout every stage of this project - not as a shortcut, but as a force multiplier for reasoning, iteration, and implementation speed.

| Stage | AI Tool Used | How It Helped |
|---|---|---|
| Schema Analysis | Claude (conversation) | Decomposed the target schema into extraction complexity levels. Identified which fields need inference vs. explicit extraction. |
| Architecture Design | Claude (conversation) | Iterated on pipeline node design. Debated trade-offs (monolithic prompt vs. multi-node). Decided on LangGraph DAG approach. |
| Implementation | Claude Code | Scaffolded LangGraph pipeline, Pydantic models, Streamlit app. Rapid iteration on code structure. |
| Prompt Engineering | Claude + manual iteration | Tested different prompt structures. Key insight: schema-first prompts with calibration examples outperform narrative prompts. |
| Edge Case Handling | Claude (conversation) | Designed conditional save detection as a universal pattern after identifying the SF-2026-0001 edge case. Ensured no hardcoding. |
| Presentation Prep | Claude (conversation) | Strategy for presentation structure, Q&A preparation, and this document itself. |

## 5.2 What Worked Well

- Using AI to rapidly prototype different prompt structures saved hours of manual trial-and-error. I could test 5-6 prompt variations in the time it would take to manually write and test one.
- Claude Code for scaffolding the LangGraph pipeline was extremely efficient - but required careful verification prompts to ensure no hardcoding to specific interaction IDs.
- Conversational AI for architecture design: treating Claude as a sounding board for trade-off discussions produced better decisions than designing in isolation.

## 5.3 What Didn’t Work / Lessons Learned

- Initial prompts were too verbose: the LLM would hallucinate additional fields not in the schema. Constraining to structured output mode solved this.

- First-pass code generation sometimes hardcoded logic to specific test cases (e.g., checking for interaction_id == "SF-2026-0001" instead of universal pattern matching). Needed explicit verification prompts to catch this.
- Over-reliance on AI for judgment calls: for fields like churn_risk, I needed to form my own opinion first and then validate against the LLM output, not the other way around.

## AI Workflow Evidence Placeholder

**I gave claude planning agent my raw notes and pdf and prompt to identify the gap**

Here's what I found:

---

### The 7 Gaps

**GAP 1 — Production Architecture is half-baked**
Your PDF sketches the components but the brief explicitly asks for: cloud service selection with reasoning, a cost model at scale (token costs × volume), error handling strategy, data quality edge cases (empty transcripts, PII, non-English), monitoring, and batch vs real-time decision. None of that is in your notes.

**GAP 2 — Working Extraction has no plan**
Page 1A shows the *output* but you haven't designed the *extraction prompt itself*, how to show raw LLM output side-by-side (brief explicitly asks for this), or which 5 interactions to pick (should be diverse: phone + chat + email, clean + noisy).

**GAP 3 — Analysis is vague**
Page 2 says "consolidated DataFrame + dashboards" but doesn't answer the 3 questions with specifics. Also — **Page 8 "What will DataFrame look like?" was left blank.** That's the backbone of the entire analysis layer.

**GAP 4 — AI Workflow Evidence is completely missing**
This is **mandatory** per the brief and not mentioned anywhere in your notes. You need screenshots/logs of Claude/Claude Code usage captured *as you build*, not retrofitted after.

**GAP 5 — Presentation narrative not mapped**
The brief has a strict 20-min script (2+5+5+3+3+2). None of your notes map to it. The "Reflections" section (what you'd do differently) needs honest prep.

**GAP 6 — Model benchmarking has no methodology**
Good idea on Page 10 but needs: same prompt, same 3 interactions, compare Claude vs GPT-4o vs Gemini on accuracy + cost + latency.

**Then I explained further**

now give me a final plan for review where you list down the componenets and how these connect and what would each component do.
for stream lit pages let me give some skeleton
page 1 - Tiles of conversation files (stored in memory) on which when i hover my cursor shows so summary using an LLM call in the backend. And when the tile is clicked opens up page 1.A. the page 1 should also contain a analyse all text file button which opens up the page 2
page 1.A - is the detail chat analysis based on the target Json schema, in the backend we need to pouplate the json target schema with insights from the txt conversations. there should be a small window like thing that shows the full populated targeted JSon schema and you will suggest what exactly the detailed analysis will tell. like identification of core crux of the problem and what other points were touhched upon in the conversation, assigning score on the agent quality and conversation quality, how complete the solution was, what the sentiment of the customer was, a full spec sheet of agent eval and customer satisfaction, you may figure a way to either call an llm for directry reading the text files or maybe use an llm to enforce populating the json targeted schema and then use it for making this elaborate spec sheet.
Page 2: this targeted Json complete schema then will be used to make a pandas dataframe, we will reverse engineer how multiple visuals and charts for the page 2 and according to that we will make a data frame from this json, we can think of a few ways to add columns if they dont exist by either giving score etc.
but i need this page to be as granularly detailed analysis as it can be, and this page should have a button which will open up page 3 which will be the remediation and action taking/suggesting page.
Page 3: LLM recommendor, this page will draw conclusion from all the poupulated json and also from pandas df and visuals which will tell the business stake holder to take actions, we will identify from the sources mentioned earlier any patterns or trends, like telling to improve tone of the human agent, or telling from backed data that we should promote a prod or a service more and if we should discontinue.
it should majorly answer these 3 questions
"What are the top drivers of customer churn, and which are most actionable?
Which customer segments show the highest upsell or retention opportunity?

**This is the plan that was reviewed by me**

*Thinking*

**Update Todos**

- [x] Create requirements.txt, .env.example, __init__.py files
- [ ] Build pipeline/models.py — Pydantic models + PipelineState
- [ ] Build pipeline/prompts.py — all LLM prompts
- [ ] Build pipeline/loader.py — file reader + channel detector
- [ ] Build pipeline/graph.py — LangGraph extraction graph
- [ ] Build analysis/dataframe.py — JSON → flat DataFrame
- [ ] Build analysis/charts.py — 12 Plotly charts
- [ ] Build streamlit_app.py — root entry point + router
- [ ] Build app/page1_tiles.py — conversation tile grid
- [ ] Build app/page1a_detail.py — deep analysis spec sheet
- [ ] Build app/page2_dashboard.py — consolidated dashboard
- [ ] Build app/page3_recommender.py — LLM recommender

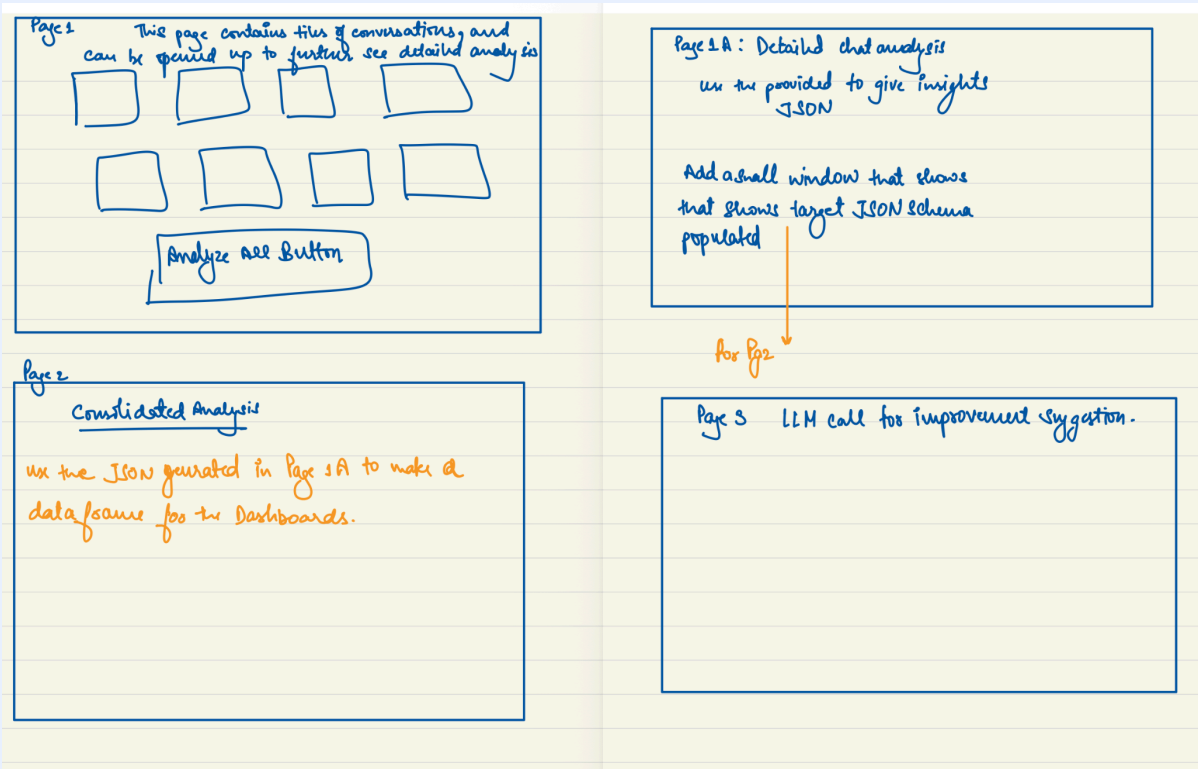
**Me checking all the components before moving forwards**

**Update Todos**

- ✓ ~~Create requirements.txt, .env.example, __init__.py files~~
- ✓ ~~Build pipeline/models.py — Pydantic models + PipelineState~~
- ✓ ~~Build pipeline/prompts.py — all LLM prompts~~
- ✓ ~~Build pipeline/loader.py — file reader + channel detector~~
- ✓ ~~Build pipeline/graph.py — LangGraph extraction graph~~
- ✓ ~~Build analysis/dataframe.py — JSON → flat DataFrame~~
- ✓ ~~Build analysis/charts.py — 12 Plotly charts~~
- ✱ Build streamlit_app.py — root entry point + router
- Build app/page1_tiles.py — conversation tile grid
- Build app/page1a_detail.py — deep analysis spec sheet
- Build app/page2_dashboard.py — consolidated dashboard
- Build app/page3_recommender.py — LLM recommender

**Write** `streamlit_app.py`

115 lines



① Database → Conversations stored

Components → ② Pulling and feeding to LLM: Input token
Transcripts will need token. output token
Thinking tokens

③ JSON →

Our future recommendation

③ Training a NLP Model for Longterm Model.

Edge Cases ↓

when working with unstructured data
→ we need to find to structure it to use it downstream.

# 6. Reflections

## 6.1 What I Would Do Differently

1. Involve multiple human annotators for ground truth creation. Currently the ground truth was generated by an independent LLM pass  effective for catching systematic failures like trajectory inflation and resolution optimism bias, but a production system needs 2-3 human annotators with inter-rater agreement scoring to establish a reliable gold standard.

2. Implement prompt versioning earlier. As prompts evolved, I lost track of which version produced which results. A prompt registry (even a simple JSON file with version numbers and timestamps) would have helped.

3. Spend more time on the tiered extraction approach. For a production system, splitting fields into "easy extraction" and "hard inference" categories and routing them to different models would meaningfully reduce cost.

4. Add targeted calibration examples for the weakest fields. The evaluation revealed that sentiment trajectory (33% accuracy) and emotional intensity (50%) are the weakest extraction fields. Rather than rewriting the entire prompt, I would add specific calibration anchors for these fields  for example, "stable means no meaningful shift even if the tone is generally positive throughout; improving requires a clear negative-to-positive arc."

## 6.2 What's Missing

1. A production-grade evaluation pipeline  the current eval compares against single-annotator ground truth. At scale, this needs multi-annotator consensus, automated regression testing when prompts change, and per-field accuracy tracking over time.

2. Confidence scores per field : the LLM should output how certain it is about each inferred value.

3. Feedback loop : human corrections should feed back into prompt refinement automatically.

4. Multi-turn extraction : for very long transcripts, a chunking strategy with context carryover.

## 6.3 Next Steps

1. Integrate with a data warehouse (BigQuery/Snowflake) for longitudinal analysis across thousands of interactions.

2. Build a retention risk dashboard that aggregates churn signals per customer across all their interactions, not just individual calls.

3. Implement the tiered model approach and measure cost savings against quality trade-offs.

4. Add a human-in-the-loop review interface where flagged extractions can be corrected and those corrections feed back into the system.

---

## Thank you - Questions?

Karan Bhutani